Métodos Computacionais da Física B

Aluno: Natália Capra Ferrazzo - Matrícula: 00243977
IF-UFRGS

11 de abril de 2019

Resumo

Este trabalho contém uma análise sobre os métodos númericos de Euler implícito e Runge-Kutta de 2º e 4º ordem para resolução da Lei de Resfriamento de Newton.

1 Introdução

Na investigação de um homicídio, ou de uma morte acidental, é, muitas vezes, importante estimar o instante da morte. Há uma forma matemática que pode ser usada para este problema. A partir de observações experimentais, sabe-se que, com uma exatidão satisfatória em muitas circunstâncias, a temperatura superficial do corpo se altera com uma taxa proporcional à diferença de temperatura entre o corpo e o meio-ambiente. É o que se conhece como Lei do Resfriamento de Newton.

2 Método

A lei de resfriamento de Newton diz que a taxa de variação de temperatura $T(t)$ de um corpo em resfriamento é proporcional à diferença entre a temperatura do corpo e a temperatura constante T_a do meio ambiente, isto é:

$$\frac{dT}{dt} = -k.(T - T_a) \tag{1}$$

Onde:
T: temperatura do corpo no instante de tempo t
T_a: temperatura constante do ambiente
k: constante de proporcionalidade positiva que depende do material que constituí o corpo, sendo que o sinal negativo indica que a temperatura do corpo está diminuindo com o passar do tempo, em relação à temperatura do meio ambiente.

Para ilustrar os métodos numéricos utilizados neste trabalho, será utilizado o problema a seguir:

Problema Suponha que um cadáver seja encontrado em condições suspeitas. A temperatura do corpo é medida imediatamente pelo perito e o valor obtido é $T = 30^oC$. A perícia conclui que o crime ocorreu há uma hora e que a temperatura do meio ambiente entre a hora da morte T_a e a hora em que o cadáver foi encontrado tenha se mantido mais ou menos constante ($T \cong 20^oC$). A perícia sabe também que a temperatura normal de um ser humano vivo é de 37 °C. O corpo é retirado da cena crime e duas horas depois, quando chegou ao necrotério, sua temperatura é novamente medida. Qual o valor esperado para essa nova medida?

2.1 Resolução analítica

$$\frac{dT}{dt} = -k.(T - T_a) \tag{2}$$
$$\frac{dT}{(T - T_a)} = -k.dt \tag{3}$$
$$\int \frac{dT}{(T - T_a)} = \int -k.dt \tag{4}$$
$$ln(T - T_a) = -kt + C \tag{5}$$
$$T - T_a = e^{-kt+C} \tag{6}$$
$$e^C = C_1 \tag{7}$$
$$\mathbf{T = C_1 e^{-kt} + T_a} \tag{8}$$

Aplicando as condições inicias ($t_0 = 0$, $T = 37^oC$), podemos encontrar a constante de resfriamento C_1:

$$37 = C_1 e^{-k.0} + 20 \tag{9}$$
$$37 - 20 = C_1 \tag{10}$$
$$C_1 = 17 \tag{11}$$

Aplicando as condições em que o corpo foi encontrado ($t = 1h$, $T = 30^oC$), consegue-se determinar a constante de proporcionalidade k:

$$T = 17e^{-kt} + 20 \tag{12}$$
$$30 = 17e^{-k1} + 20 \tag{13}$$
$$30 - 20 = 17e^{-k} \tag{14}$$
$$e^{-k} = \frac{10}{17} \tag{15}$$
$$ln(e^{-k}) = ln(0,588) \tag{16}$$
$$-k = ln(0,588) \tag{17}$$
$$\mathbf{k \cong 0,53h^{-1}} \tag{18}$$

Portanto, a equação que descreve a Temperatura do corpo em função do tempo é:

$$\mathbf{T = 17e^{-0,53t} + 20} \tag{19}$$

Para determinar a temperatura em que o corpo chegou ao necrotério, temos:

$$T = 17e^{-0,53.3} + 20 \tag{20}$$

$$\mathbf{T \cong 23,5^{o}C} \tag{21}$$

2.2 Resolução Numérica

Nas resoluções numéricas, utiliza-se métodos de aproximação a fim de obter o resultado mais próximo da resolução analítica. Abaixo, serão apresentados três métodos de integração numérica que posteriormente serão comparados graficamente com os resultados obtidos por meio da resolução analítica. Para todos, as condições iniciais usadas foram:

$$t_0 = 0; t_f = 3; T_0 = 37^{o}C; T_a = 20^{o}C; k = 0,530628251 \tag{22}$$

Aplicando estes valores mais precisos, de forma analítica, $T_f = 21,64316767^{o}C$.

2.2.1 Método de Euler Explícito

O método de Euler Explícito é um método de intergração numérica obtido através de expansão de Série de Taylor.

É chamado de explícito pois os valores de x_{n+1} são obtidos a partir da resolução de uma equação explícita, ou seja, de uma equação que não depende de x_{n+1}.

A forma geral deste método é:

$$T_{n+1} = T_n + f(T_n, t_n)h + \varepsilon(h^2) \tag{23}$$

O termo $\varepsilon(h^2)$ significa que o método gera um erro local de ordem 2.

2.2.2 Runge-Kutta 2ª ordem

Runge-Kutta é um outro método numéridco para resolução de EDO's. Consiste em utilizarmos mais termos da expansão de x_{n+1} em séries de Taylor. Seu erro local é da ordem de h^3, gerando um erro global de 2ª ordem. O método implementado é o ponto médio.

$$k_1 = f(t_n, x_n) \tag{24}$$

$$k_2 = f\left(t_n + \frac{h}{2}, x_n + \frac{k_1 h}{2}\right) \tag{25}$$

$$x_{n+1} = x_n + k_2 h + \varepsilon(h^3) \tag{26}$$

2.2.3 Runge-Kutta 4ª ordem

Este método, chamado de RK4, utiliza as 5 primeiras parcelas da série de Taylor. Seu erro global é da ordem de h^4.

$$k_1 = f(t_n, x_n) \tag{27}$$

$$k_2 = f\left(t_n + \frac{1}{2}h, x_n + \frac{k_1 h}{2}\right) \tag{28}$$

$$k_3 = f\left(t_n + \frac{1}{2}h, x_n - \frac{k_2 h}{2} + k_2 h\right) \tag{29}$$

$$k_4 = f\left(t_n + h, x_n + k_3 h\right) \tag{30}$$

$$x_{n+1} = x_n \frac{1}{6}\left[k_1 + 2k_2 + 2k_3 + k_4\right] h + \varepsilon(h^5) \tag{31}$$

3 Resultados

3.1 Análise dos Métodos

No gráfico a seguir podemos ver uma comparação entre a resolução analítica e os 3 métodos numéricos.

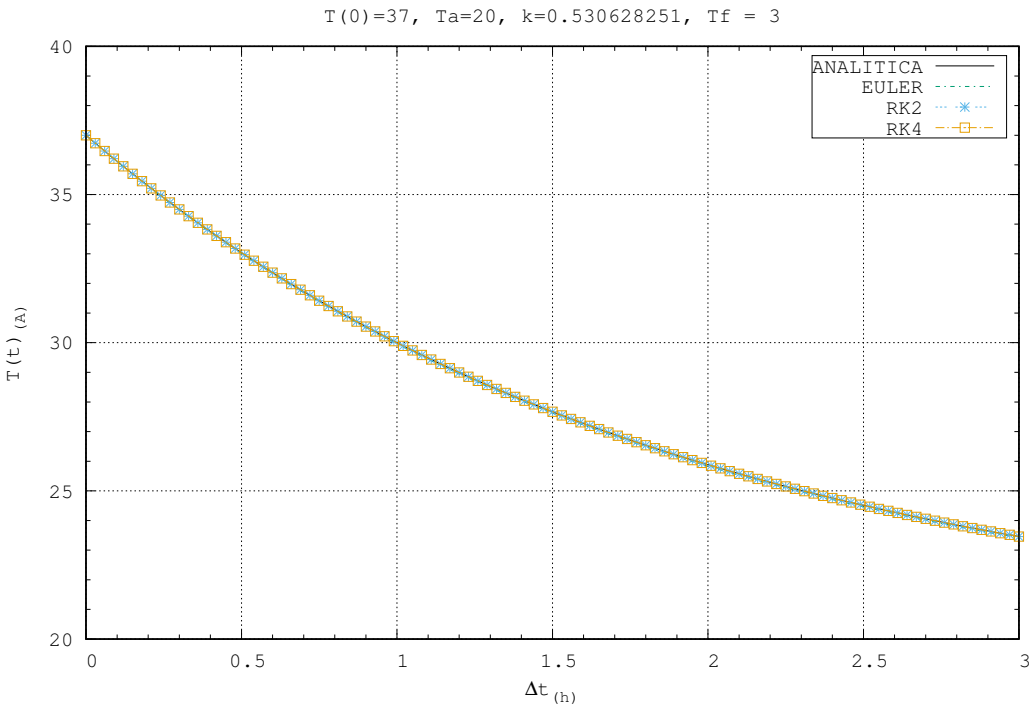

Figura 1: Valores numéricos X Valor analítico

Abaixo, o gráfico em detalhe.

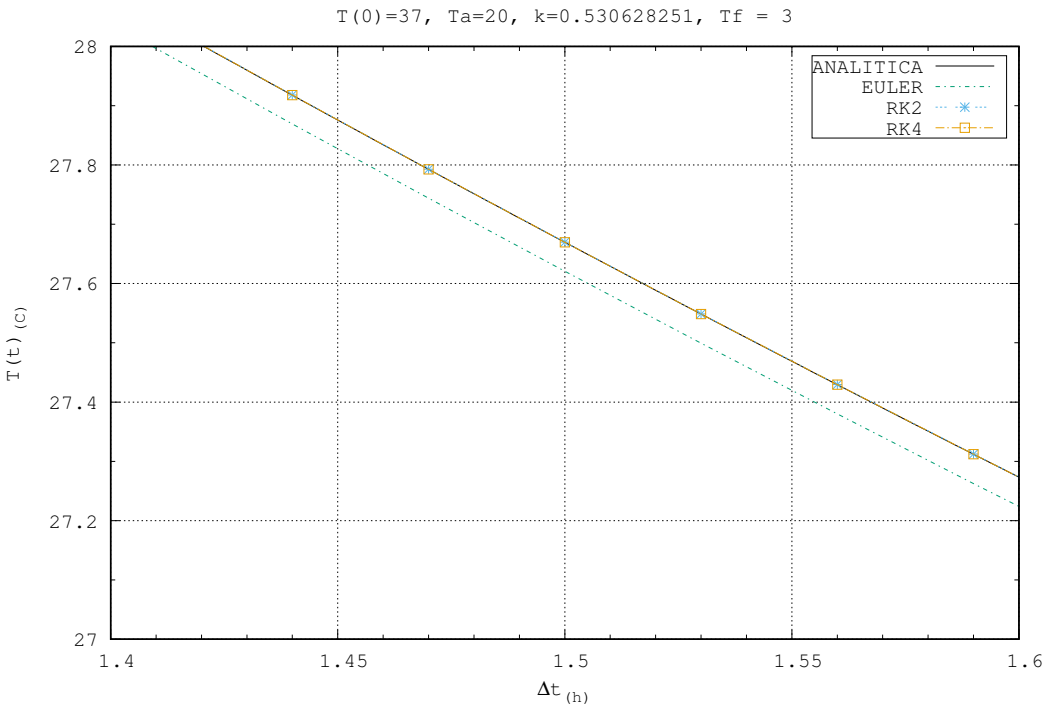

Figura 2: Diferença dos métodos

3.2 Erros Relativos

Para calcular os erros numéricos foi utilizado, em cada caso, a equação de Erro Relativo, dada a seguir:

$$ErroRelativo(\%) = \left| \frac{F(x) - N(x)}{F(x)} \right| \tag{32}$$

Onde:

- $F(x) \equiv$ Resolução analítica;
- $N(x) \equiv$ Resolução numérica.

No gráfico abaixo, podemos ver como os 3 métodos se comportam a medida em que o número de passos N aumenta.

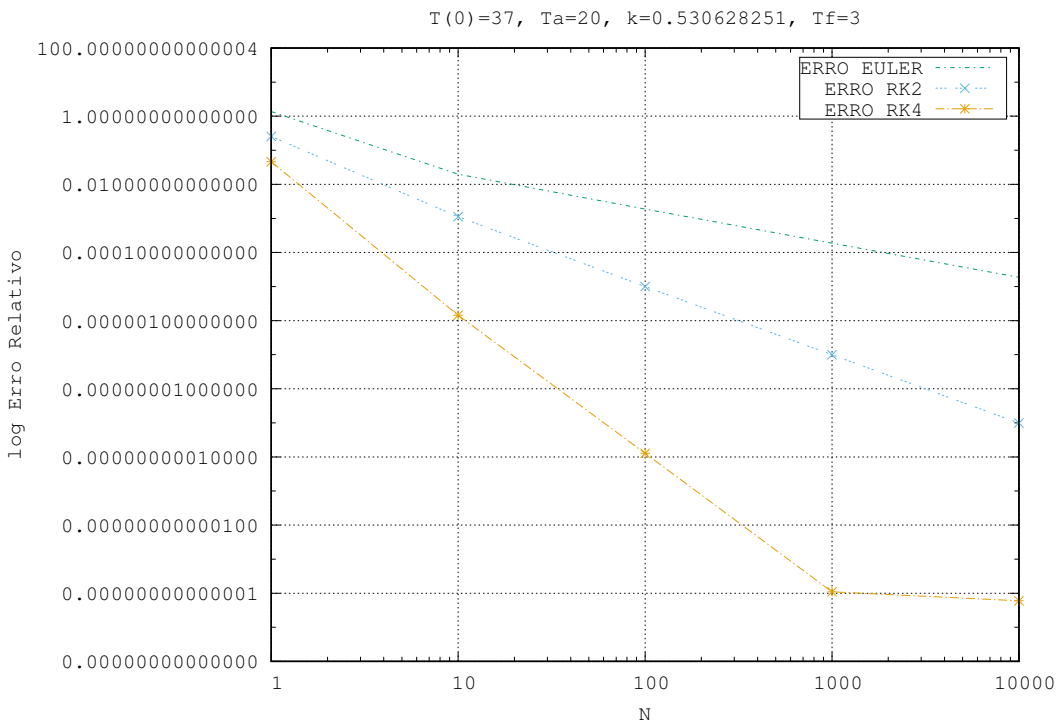

Figura 3: Este gráfico nos mostra a ordem de erro de cada método

Se calcularmos a derivada das curvas acima, veremos como o erro de Euler Explícito tem ordem 1, o de Runge-Kutta de 2ª Ordem, de ordem 2, e o Runge-Kutta de 4ª ordem, de ordem 4. Sendo daí que vem o nome destes dois últimos citados.

4 Código-Fonte

4.1 Euler

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
# *** template python ***

from math import *

# CONDIÇÕES INICIAIS #
t0=0.
tf=3.
T0=37.
Tamb=20.
k = 0.530628251
N = 100
h = (tf-t0)/N

t = t0
T = T0
Ta = T0

#IMPRIME OS VALORES INICIAIS#
print '%.15f' % t, '%.15f' %  T,  '%.15f' % Ta


# FUNÇÃO ANALÍTICA #
def T_analitic(t):
return (17*(e**(-k*t))+20)

# FUNÇÃO #
def knum(T,t):
return ((-k*(T-Tamb)))

# LAÇO DE ITERAÇÃO #
for i in range (1,N+1):

t = i*h

T = T + h*knum(T,t)
```

```
Ta = T_analitic(t)

print '%.15f' % t, '%.15f' %  T, '%.15f' % Ta
```

4.2 Ponto Médio

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
# *** template python ***

from math import *

# CONSTANTES DO MÉTODO #
a2 = 1.
a1 = 1.-a2
p1 = 1./(2.*a2)
q11 = 1./(2.*a2)

# CONDIÇÕES INICIAIS #
t0=0.
tf=3.
T0=37.
Tamb=20.
k = 0.530628251
N = 100
h = (tf-t0)/N

k1, k2 = 0, 0

t = t0
T = T0
Ta = T0

# IMPRIME OS VALORES INICIAIS #
print '%.15f' % t, '%.15f' %  T,  '%.15f' % Ta

# FUNÇÃO ANALÍTICA #
def T_analitic(t):
return (17*(e**(-k*t))+20)
```

```
# FUNÇÃO GERADORA DOS k'S #
def knum(T,t):
return ((-k*(T-Tamb)))

# LAÇO DE ITERAÇÃO #
for i in range (1,N+1):

t = i*h

k1 = knum(T, t)
k2 = knum(T + q11*k1*h, t + p1*h)

T = T + h*(a1*k1+a2*k2)

Ta = T_analitic(t)

print '%.15f' % t, '%.15f' %  T, '%.15f' % Ta
```

4.3 RK4

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
# *** template python ***

from math import *

# CONDIÇOES INICIAIS #
t0=0.
tf=3.
T0=37.
Tamb=20.
k = 0.530628251
N = 100
h = (tf-t0)/N

k1, k2, k3, k4 = 0, 0, 0, 0

t = t0
T = T0
```

```
Ta = T0

# IMPRIME OS VALORES INICIAIS #
print '%.15f' % t, '%.15f' %  T,  '%.15f' % Ta

# FUNÇÃO ANALÍTICA #
def T_analitic(t):
return (17*(e**(-k*t))+20)

# FUNÇÃO GERADORA DOS k'S #
def knum(T,t):
return ((-k*(T-Tamb)))

# LAÇO DE ITERAÇÃO #
for i in range (1,N+1):

t = i*h

k1 = h*knum(T, t)
k2 = h*knum(T + (1./2.)*k1, t + (1./2.)*h)
k3 = h*knum(T + (1./2.)*k2, t + (1./2.)*h)
k4 = h*knum(T + k3, t + h)

T = T + (1./6.)*k1 + (1./3.)*k2 + (1./3.)*k3 + (1./6.)*k4

Ta = T_analitic(t)

print '%.15f' % t, '%.15f' %  T, '%.15f' % Ta
```

4.4 Euler - Erro

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
# *** template python ***

from math import *

# CONDIÇÕES INICIAIS #
t0=0.
tf=3.
```

```
T0=37.
Tamb=20.
k = 0.530628251
N = 1
h = (tf-t0)/N

t = t0
T = T0
Ta = T0

# FUNÇÃO ANALÍTICA #
def T_analitic(t):
return (17*(e**(-k*t))+20)

# FUNÇÃO #
def knum(T,t):
return ((-k*(T-Tamb)))

# LAÇO DE ITERAÇÃO #
while (N<=10000):

h = (tf-t0)/N
t = t0
T = T0
Ta = T0

for i in range (1,N+1):

t = i*h

T = T + h*knum(T,t)

Ta = T_analitic(t)

erro = (Ta - T)/T

print '%.f' % N, '%.15f' %  erro

N = N*10
```

4.5 Ponto Médio - Erro

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
# *** template python ***

from math import *

# CONSTANTES DO MÉTODO #
a2 = 1.
a1 = 1.-a2
p1 = 1./(2.*a2)
q11 = 1./(2.*a2)

# CONDIÇÕES INICIAIS #
t0=0.
tf=3.
T0=37.
Tamb=20.
k = 0.530628251
N = 1
h = (tf-t0)/N

k1, k2 = 0, 0

t = t0
T = T0
Ta = T0

# FUNÇÃO ANALÍTICA #
def T_analitic(t):
return (17*(e**(-k*t))+20)

# FUNÇÃO GERADORA DOS k'S #
def knum(T,t):
return ((-k*(T-Tamb)))

# LAÇO DE ITERAÇÃO #

while (N<=10000):
```

```
h = (tf-t0)/N
t = t0
T = T0
Ta = T0

for i in range (1,N+1):

t = i*h

k1 = knum(T, t)
k2 = knum(T + q11*k1*h, t + p1*h)

T = T + h*(a1*k1+a2*k2)

Ta = T_analitic(t)

erro = abs((Ta - T)/T)

print '%.f' % N, '%.15f' %  erro

N = N*10
```

4.6 RK4 - Erro

```
#!/usr/bin/python
# -*- coding: utf-8 -*-
# *** template python ***

from math import *

# CONDIÇOES INICIAIS #
t0=0.
tf=3.
T0=37.
Tamb=20.
k = 0.530628251
N = 1
h = (tf-t0)/N

k1, k2, k3, k4 = 0, 0, 0, 0
```

```
t = t0
T = T0
Ta = T0

# FUNÇÃO ANALÍTICA #
def T_analitic(t):
return (17*(e**(-k*t))+20)

# FUNÇÃO GERADORA DOS k'S #
def knum(T,t):
return ((-k*(T-Tamb)))

# LAÇO DE ITERAÇÃO #
while (N<=10000):

h = (tf-t0)/N
t = t0
T = T0
Ta = T0

for i in range (1,N+1):

t = i*h

k1 = h*knum(T, t)
k2 = h*knum(T + (1./2.)*k1, t + (1./2.)*h)
k3 = h*knum(T + (1./2.)*k2, t + (1./2.)*h)
k4 = h*knum(T + k3, t + h)

T = T + (1./6.)*k1 + (1./3.)*k2 + (1./3.)*k3 + (1./6.)*k4

Ta = T_analitic(t)

erro = abs((Ta - T)/T)

print '%.f' % N, '%.15f' %  erro

N = N*10
```

5 Scripts

5.1 Análise

```
    ##### RESOLUÇAO EDO #####
reset #Reseta o gnuplot
set xrange[0:3] #Define range do eixo x
set yrange[20:40] #Define range do eixo y
set grid #Seta grades
set xtics #Seta pontos na linha do eixo x
set ytics #Seta pontos na linha do eixo y
set mxtics #Seta pontos intermediarios em x
set mytics #Seta pontos intermediarios em y
set key box r t #Seta legendas com caixa no canto sup. dir.
plot ((17)*(exp(-0.530628251*(x)))+20) w l dt 1 lc -1 t 'ANALITICA'
#Plota funçao
#w= escrever; #l= linha; #dt= tipo de linha; #lp= lin. pto; #pt= pto;
#lc= cor; #t= titulo
replot 'Euler.dat' u 1:2 w l dt 8 lc 2 t 'EULER'
replot 'pontomedio.dat' u 1:2 w lp dt 7 lc 3 t 'RK2'
replot 'RK4.dat' u 1:2 w lp dt 13 lc 4 t 'RK4'
set xlabel '{/Symbol D}t_{(h)}' #Define titulo do eixo x
set ylabel 'T(t)_{(C)}' #Define titulo do eixo y
set title 'T(0)=37, Ta=20, k=0.530628251, Tf = 3' #Define titulo
set term pos eps enh col "ascii" 14 #Define fomato de saida
set out "grafico.eps" #Define o nome do arquivo salvo
replot #Cria a figura
set xrange[1.4:1.6] #Define range do eixo x
set yrange[27:28] #Define range do eixo y
set term pos eps enh col "ascii" 14 #Define fomato de saida
set out "graficozoom.eps" #Define o nome do arquivo salvo
replot #Cria a figura
unset out #Volta a ediçao de gnuplot anterior
unset term #Volta a ediçao de gnuplot anterior
```

5.2 Erro

```
    ##### ERRO #####
reset #Reseta o gnuplot
set log
set grid #Seta grades
set xtics #Seta pontos na linha do eixo x
set ytics #Seta pontos na linha do eixo y
set mxtics #Seta pontos intermediarios em x
set mytics #Seta pontos intermediarios em y
set key box r t #Seta legendas com caixa no canto sup. dir.
plot 'erroeu.dat' u 1:2 w l dt 8 lc 2 t 'ERRO EULER'
replot 'erropm.dat' u 1:2 w lp dt 7 lc 3 t 'ERRO RK2'
replot 'erroRK4.dat' u 1:2 w lp dt 13 lc 4 t 'ERRO RK4'
set xlabel '{N}' #Define titulo do eixo x
set ylabel 'log Erro Relativo' #Define titulo do eixo y
set title 'T(0)=37, Ta=20, k=0.530628251, Tf=3' #Define titulo
set term pos eps enh col "ascii" 14 #Define fomato de saida
set out "erro.eps" #Define o nome do arquivo salvo
replot #Cria a figura
unset out #Volta a ediçao de gnuplot anterior
unset term #Volta a ediçao de gnuplot anterior
```

Referências

[1] Hentz, A.; Brunnet, L.; Fernandes, H. *Métodos Computacionais da Física B*. Instituto de Física - Universidade Federal do Rio Grande do Sul, Porto Alegre - RS.

[2] Resfriamento de Newton. In: FUCAP. Disponível em: http://www.fucap.edu.br/portal/artigos/Artigoequacoesdiferenciais.pdf. Acesso em: 10 abr. 2019.

[3] Justo, D. A.; Sauter, E.; Azevedo, F.; Guidi, L.; Konzel, P. H. *Cálculo numérico, um livro colaborativo*. Instituto de Matemática - Universidade Federal do Rio Grande do Sul, Porto Alegre - RS, 2019.